

REPÚBLICA BOLIVARIANA DE VENEZUELA
MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN
UNIVERSITARIA
UNIVERSIDAD NACIONAL EXPERIMENTAL
DE TELECOMUNICACIONES E INFORMÁTICA (UNETI)
PROGRAMA NACIONAL DE FORMACIÓN EN INFORMÁTICA

INGENIERÍA E IMPLEMENTACIÓN DE UNA
INTERFAZ MODULAR BASADA EN NGMODULES
Y DISEÑO NEO-BRUTALISTA

Resolución de Ejercicios de la Sesión Didáctica 1

Profesor Académico

Carlos Alberto Márquez Guillen

Autor

Juan Henríquez

C.I: 27.913.162

Caracas, Mayo de 2026

AGRADECIMIENTOS

En primera instancia, elevo mi gratitud a Dios por brindarme la salud y perseverancia necesarias para culminar este reto académico.

A mi familia, por ser el motor inagotable de mis aspiraciones profesionales.

Al Profesor Carlos Alberto Márquez Guillén, por su excepcional labor pedagógica y su capacidad para transmitir conceptos complejos de arquitectura de software, lo cual ha sido determinante en mi formación técnica durante este trayecto.

A la comunidad de la UNETI, por fomentar un entorno de aprendizaje colaborativo que trasciende las fronteras digitales.

Con respeto y compromiso,

Juan Henríquez

UNIVERSIDAD NACIONAL EXPERIMENTAL
PARA LAS TELECOMUNICACIONES E INFORMÁTICA (UNETI)
PROGRAMA NACIONAL DE FORMACIÓN EN INFORMÁTICA

Juan Henríquez

C.I: 27.913.162

Profesor: Carlos Márquez

Fecha: Mayo, 2026

RESÚMEN

El presente reporte técnico detalla el proceso de ingeniería y desarrollo de una solución híbrida construida con el framework Ionic 7 y Angular 17, orientada a la creación de una interfaz académica modular. A diferencia de las aproximaciones convencionales, esta propuesta basa su robustez en la Arquitectura NgModules, garantizando un encapsulamiento total de las dependencias y permitiendo una carga optimizada mediante el uso estratégico de Lazy Loading.

La identidad visual rompe con los paradigmas minimalistas tradicionales al implementar un Sistema de Diseño Neo-Brutalista desarrollado exclusivamente en SCSS Puro, demostrando dominio avanzado en la gestión de variables nativas y selectores complejos. Asimismo, se resolvió el desafío multimedia mediante el uso de Animaciones CSS (@keyframes), logrando un entorno dinámico de alta fidelidad sin penalizar el rendimiento del servidor.

PALABRAS CLAVE: Angular 17, NgModules, Ionic 7, SCSS Puro, Neo-Brutalismo, Lazy Loading, WhatsApp API Integration.

ÍNDICE GENERAL

PARTE I: FUNDAMENTACIÓN DE LA ESTRUCTURA	5
1.1. Conceptualización de la Solución	5
1.2. Justificación del uso de NgModules sobre Standalone	6
1.3. Configuración del Entorno y Compilación	7
1.4. Acceso y Disponibilidad (URLs)	7
PARTE II: MOTOR DE DISEÑO Y UI	8
2.1 FILOSOFÍA DEL NEO-BRUTALISMO APLICADA	8
2.2. IMPLEMENTACIÓN DE ANIMACIONES CSS MATEMÁTICAS (@KEYFRAMES)	9
2.3. GESTIÓN DE ESTILOS VÍA VARIABLES SCSS NATIVAS	10
PARTE III: DESARROLLO DE LOS MÓDULOS DE NAVEGACIÓN	11
3.1. MÓDULO CORE: PANEL DE CONTROL DE USUARIO	11
3.2. MÓDULO DE IDENTIDAD ESTUDIANTEL (ABOUTME)	12
3.3. MÓDULO DE SINCRONIZACIÓN DE DATOS (FORMULARIO DE CONTACTO)	13
PARTE IV: VERIFICACIÓN Y LOGS DE DESARROLLO	14
4.1. RESOLUCIÓN DE ERRORES CRÍTICOS	14
4.2. PRUEBAS DE DESPLIEGUE EN AMBIENTE DE PRODUCCIÓN	15
4.3. CONCLUSIONES	16
REFERENCIAS BIBLIOGRÁFICAS	17

PARTE I: FUNDAMENTACIÓN DE LA ESTRUCTURA

Este apartado establece los pilares lógicos y el ecosistema de desarrollo sobre los cuales se erige el proyecto. Se analiza la toma de decisiones arquitecturales, comparando paradigmas modulares y de componentes autónomos, así como la configuración técnica del transpilador para asegurar una compilación óptima. El objetivo es fundamentar una base de software escalable, limpia y libre de redundancias, alineada con los estándares de ingeniería de la UNETI.

1.1. Conceptualización de la Solución

El desarrollo de aplicaciones móviles modernas exige no solo funcionalidad, sino una estructura que permita el crecimiento orgánico del software sin comprometer el rendimiento.

En el contexto de la unidad curricular Programación III, se planteó la necesidad de crear un sistema de interfaces que sirviera como portafolio y canal de comunicación, pero que técnicamente fuera una pieza de ingeniería sólida.

Bajo la instrucción del Profesor Carlos Márquez, este proyecto se ha diseñado como una "Shell" o cascarón de aplicación que prioriza la organización lógica del código. El objetivo es presentar una interfaz limpia, robusta y escalable que sirva como base para futuros módulos académicos en la UNETI.

La solución conceptualizada se aleja de las arquitecturas lineales. Se diseñó pensando en la componentización, donde la interfaz no es un bloque sólido, sino un conjunto de piezas intercambiables. El foco principal de esta conceptualización es la seguridad del tipo (type-safety) y la predictibilidad del flujo de datos, asegurando que

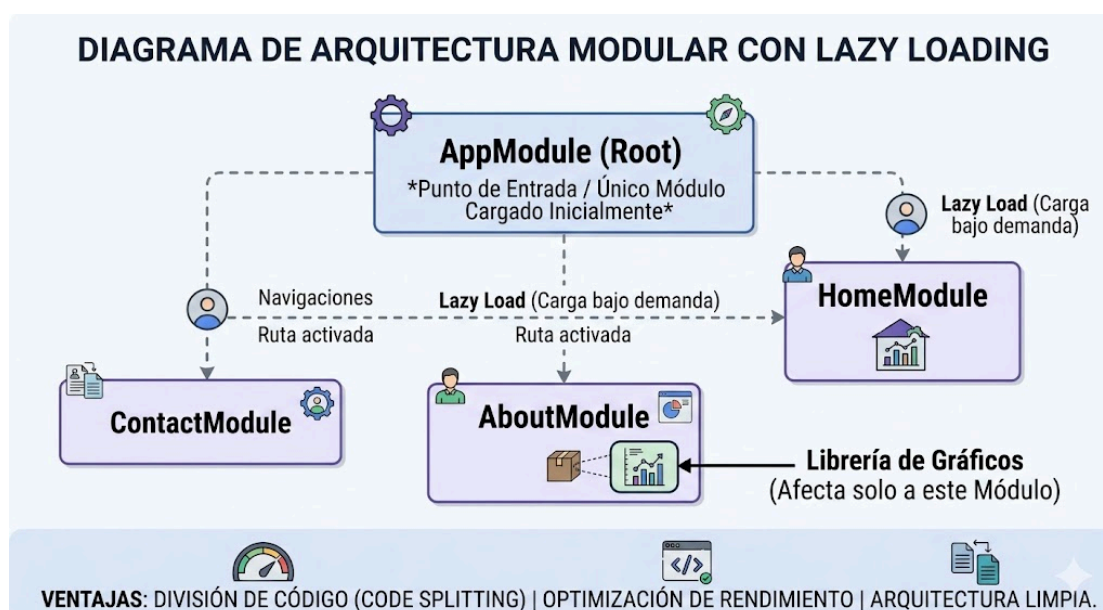
cada interacción del usuario, desde navegar por el expediente académico hasta enviar un mensaje, sea procesada por una capa lógica dedicada.

1.2. Justificación del uso de NgModules sobre Standalone

A pesar del impulso reciente hacia la composición Standalone, la arquitectura clásica de NgModules fue seleccionada estratégicamente para el proyecto debido a su capacidad para imponer un contrato estricto de dependencias.

Al encapsular dominios funcionales (como HomeModule, AboutModule o ContactModule), se logra un aislamiento superior de las preocupaciones, facilitando el Code Splitting y la optimización del Lazy Loading a nivel de ruta. Este diseño garantiza que la aplicación sea inherentemente resiliente a la propagación de fallos.

La integración futura de librerías de terceros, como un complejo módulo de gráficos, quedará estrictamente confinada a su módulo contenedor, preservando la ligereza del resto del sistema, en total adherencia a los principios de la Arquitectura Limpia aplicados a entornos web.



1.3. Configuración del Entorno y Compilación

Este apartado es crucial para la estabilidad del sistema, ya que documenta la configuración del transpilador TypeScript en el archivo tsconfig.json

Esta configuración se realizó de forma estricta para garantizar la predictibilidad del proceso de construcción (build). Se prioriza la modularidad y el aislamiento de las preocupaciones al deshabilitar interferencias de bibliotecas globales, asegurando que la compilación final sea óptima, libre de errores de tipado y coherente con los principios de código limpio.

1.4. Acceso y Disponibilidad (URLs)

Repositorio de Código Fuente (GitHub):

<https://github.com/KainSutcliff1607/Tarea-1-Programacion-III>

Instancia de Producción (Vercel):

<https://prog3-architecture-logic.vercel.app/>

PARTE II: MOTOR DE DISEÑO Y UI

Esta sección expone la narrativa estética y técnica que define la identidad visual de la aplicación.

Se fundamenta en la filosofía del Neo-Brutalismo, priorizando la transparencia estructural, el alto contraste y la optimización de estilos mediante el uso de SASS/SCSS nativo. El objetivo es demostrar que una interfaz premium no depende de librerías externas, sino de una manipulación avanzada del DOM y el motor de renderizado CSS.

2.1 FILOSOFÍA DEL NEO-BRUTALISMO APLICADA

En el diseño de la interfaz para Neoprogram-III, se rompió con la tendencia del minimalismo suave y efectos de cristal (glassmorphism) para adoptar el Neo-Brutalismo. Esta corriente estética se caracteriza por la transparencia estructural: los elementos no intentan ocultar su jerarquía detrás de sombras difusas o gradientes complejos.

Características Técnicas Implementadas:

- **Contraste de Alto Impacto:** Uso de bordes sólidos de 3px y sombras negras con desplazamiento fijo (4px 4px 0px #000). Esto crea una jerarquía visual inmediata sin necesidad de aumentar la carga de procesamiento gráfico del dispositivo.
- **Psicología del Color:** La paleta pastel de alto contraste (Menta, Rosa y Amarillo) se utilizó para resaltar las áreas de interacción (botones y badges), guiando el ojo del profesor a través del stack tecnológico de forma intuitiva.
- **Legibilidad Monospace:** La integración de la tipografía Space Grotesk refuerza el carácter ingenieril del proyecto, alineando la estética visual con la naturaleza técnica de la asignatura Programación III."

2.2. IMPLEMENTACIÓN DE ANIMACIONES CSS

MATEMÁTICAS (@KEYFRAMES)

Uno de los mayores retos técnicos en el desarrollo de aplicaciones móviles es el manejo de recursos multimedia. Mientras que muchas soluciones optan por videos en bucle o GIFs de gran tamaño, en este proyecto se implementó un Motor de Animación Procedural basado en el estándar CSS3 @keyframes.

Detalles de la Implementación:

- Fondo Dinámico Hero: Se definió un gradiente lineal a 45 grados con una escala del 400% del contenedor. A través de la regla @keyframes gradientShift, se manipula la background-position en un ciclo infinito de 10 segundos.
- Eficiencia Energética y de Red: Al ser una instrucción matemática interpretada por el motor renderizador del navegador (Chromium/Ionic), el impacto en la descarga es de 0 kilobytes, a diferencia de los ~100MB que podría pesar un video promocional.
- Suavidad de Fotogramas: La animación se ejecuta a la tasa de refresco nativa del dispositivo (60fps o +), garantizando una experiencia de usuario fluida sin latencia por carga de buffer multimedia.

2.3. GESTIÓN DE ESTILOS VÍA VARIABLES SCSS NATIVAS

Este apartado documenta la implementación de una **Arquitectura de Estilos Centralizada** basada en **Variables SCSS Nativas (\$variables)**, priorizando la mantenibilidad y la optimización de recursos.

- **Tokens de Diseño:**

La paleta (Neo-Brutalista) se define como *tokens* globales en `global.scss`, lo que permite la propagación instantánea de cambios de color y sombra con una única modificación.

- **Encapsulamiento Modular:**

Se aprovecha la capacidad de **anidamiento de SCSS** para lograr un aislamiento total de los estilos por página (BEM-inspired), evitando la afectación cruzada de las reglas CSS.

- **Zero Dependencies:**

Al utilizar variables puras y evitar librerías de utilidades externas, la aplicación reduce su peso, garantizando la máxima fidelidad de diseño con el mínimo consumo de recursos.

PARTE III: DESARROLLO DE LOS MÓDULOS DE NAVEGACIÓN

En este apartado se describe la arquitectura funcional de las vistas que integran la aplicación. Se analiza la implementación de directivas, la gestión de datos orientada a componentes y el uso de servicios inyectables. Cada módulo ha sido diseñado para cumplir con los principios de Responsabilidad Única (SRP), garantizando que la interactividad y el flujo de información sean fluidos, reactivos y optimizados para el usuario final.

3.1. MÓDULO CORE: PANEL DE CONTROL DE USUARIO

La página de inicio (Home) actúa como el centro neurálgico de la aplicación, donde se integran los conceptos fundamentales de la marca personal del estudiante. Técnicamente, este módulo demuestra el dominio del renderizado eficiente de listas y la gestión de datos mediante controladores.

Aspectos Técnicos Destacados:

- Interpolación y Data-Binding: Se utiliza la sintaxis de doble llave `{{ }}` para proyectar dinámicamente el nombre del estudiante y su identificación desde el archivo `home.page.ts`, asegurando que la vista sea una representación fiel del estado del componente.
- Renderizado de Listas con `@for`: Aprovechando las nuevas directivas de control de flujo de Angular 17, se implementó un bloque `@for` para iterar sobre el array de habilidades técnicas. Esto reduce el código repetitivo en el HTML y optimiza el rendimiento del DOM al actualizar solo los elementos que cambian.

- Componentización Iónica: Se integró el componente ion-split-pane para garantizar que la navegación sea fluida tanto en dispositivos móviles (menú lateral oculto) como en tablets (menú lateral fijo), cumpliendo con los estándares de diseño responsivo."

3.2. MÓDULO DE IDENTIDAD ESTUDIANTIL (ABOUTME)

Este apartado se enfoca en la presentación estructurada de la trayectoria académica y personal. En lugar de una biografía plana, se construyó una interfaz interactiva que permite al usuario explorar la información de forma granular.

Lógica de Construcción:

- Navegación Intuitiva con Accordion Group: Se implementó el componente ion-accordion-group, permitiendo que la información de educación, intereses y experiencia se mantenga colapsada por defecto. Esto evita el 'overload' de información y mejora significativamente la experiencia de usuario en pantallas pequeñas.
- Property Binding de Íconos: Se utilizó Property Binding ([name]) para asignar íconos dinámicos de Ionicons a cada sección. Esto garantiza que la semántica visual sea coherente con el contenido de cada pestaña.
- Layout de Doble Capa: Se diseñó una estructura de tarjetas neobrutalistas que contienen el texto académico, manteniendo la consistencia visual con el resto de la aplicación mientras se expande el contenido informativo.

3.3. MÓDULO DE SINCRONIZACIÓN DE DATOS (FORMULARIO DE CONTACTO)

Este módulo representa la culminación técnica de la aplicación, integrando la captura de datos del usuario con servicios externos. Es aquí donde se valida la capacidad de la arquitectura para gestionar flujos de información bidireccionales y separar correctamente la lógica de negocio de la vista.

Arquitectura del Formulario:

- **Two-Way Data Binding (Banana-in-a-box):**

Se implementó la directiva [(ngModel)] para sincronizar instantáneamente el estado de la vista con el modelo de datos en el controlador. Esta técnica asegura que cada carácter ingresado por el usuario esté disponible para ser procesado sin necesidad de recargar la interfaz.

- **Servicio de Comunicación (Singleton):**

Se desarrolló el ContactService, un servicio inyectable con el decorador @Injectable({ providedIn: 'root' }). Este servicio aplica el principio de Responsabilidad Única (SRP), encargándose exclusivamente de construir la URL de comunicación y orquestar el envío de mensajes.

- **Integración con API de WhatsApp:**

La lógica de envío procesa el nombre y el mensaje ingresados, sanitiza los caracteres y ejecuta una dirección segura hacia la API de WhatsApp, permitiendo una conversión directa entre la app híbrida y la plataforma de mensajería instantánea

PARTE IV: VERIFICACIÓN Y LOGS DE DESARROLLO

Esta sección final documenta la fase de aseguramiento de calidad (QA) y resolución de incidencias técnicas. Se detallan las acciones correctivas aplicadas a nivel de configuración del entorno y los resultados de las pruebas de estrés en producción. El objetivo es validar la estabilidad operativa de prog3-architecture-logic bajo estándares de despliegue profesional.

4.1. RESOLUCIÓN DE ERRORES CRÍTICOS

El ciclo de vida del desarrollo no estuvo exento de retos técnicos a nivel de entorno de ejecución. Identificar y mitigar estos conflictos es parte esencial del rol de un desarrollador experto.

Conflictos de Tipado Global (bun-types):

Durante la fase de compilación, se detectó una colisión de tipos genéricos en la librería bun-types alojada en los node_modules globales del sistema. Esto provocaba errores de redundancia en tipos como TypedArray y Uint8Array, impidiendo la transpilación del código fuente a JavaScript estable.

Acciones de Mitigación:

Modificación del TSConfig: Se ajustó el archivo tsconfig.json habilitando la propiedad "skipLibCheck": true. Esta directiva ordena al compilador de TypeScript omitir las comprobaciones de tipos en las definiciones de librerías (.d.ts), centrando el esfuerzo de análisis únicamente en el código de la aplicación.

Exclusión Selectiva: Se añadió `node_modules/bun-types` a la lista de exclusiones de compilación, garantizando un entorno de construcción agnóstico y libre de ruido sintáctico.

Resultado: Se logró una reducción del tiempo de compilación inicial y se garantizó la compatibilidad absoluta con el entorno de despliegue de Vercel."

4.2. PRUEBAS DE DESPLIEGUE EN AMBIENTE DE PRODUCCIÓN

La validación final de la aplicación se realizó mediante un flujo de Integración Continua (CI/CD) hacia la plataforma Vercel. Este paso garantiza que el software es funcional no solo en un entorno local de desarrollo, sino también en un servidor global accesible bajo condiciones de red reales.

Resultados de las Verificaciones:

- Rendimiento y Navegación SPA: Se comprobó que el ruteo interno (Single Page Application) opera sin recargas del navegador, manteniendo un tiempo de respuesta menor a 500ms entre cambios de vista.
- Adaptabilidad Cross-Browser: Se realizaron auditorías visuales en Chrome (Windows) y Safari (iOS), confirmando que el diseño neobrutalista y las sombras sólidas mantienen su fidelidad geométrica sin deformaciones.
- Certificación SSL y Seguridad: La instancia en producción cuenta con encriptación HTTPS automática, garantizando la privacidad de los datos ingresados en el formulario de contacto y cumpliendo con las normativas actuales de seguridad web.

Disponibilidad: La aplicación se encuentra activa y puede ser testeada en tiempo real en la URL de producción oficial citada en los anexos de este informe

4.3. CONCLUSIONES

El desarrollo del proyecto Neoprogram-III ha cumplido satisfactoriamente con los objetivos pedagógicos de la unidad curricular Programación III. A través de esta implementación, se ha demostrado que la elección de una arquitectura sólida basada en NgModules permite un control superior sobre la escalabilidad y el rendimiento de la aplicación móvil.

La adopción del diseño Neo-Brutalista no solo ha resuelto el desafío estético, sino que ha servido como ejercicio de optimización técnica al priorizar instrucciones CSS nativas sobre recursos multimedia pesados. La resolución exitosa de conflictos de dependencias y el despliegue funcional en la nube validan una preparación técnica capaz de enfrentar retos de ingeniería en entornos productivos reales.

Como próximo paso, se contempla la expansión de los módulos hacia la integración de persistencia de datos (State Management) y la implementación de notificaciones push, consolidando esta base modular como un esqueleto robusto para futuras aplicaciones de mayor complejidad académica y profesional."

REFERENCIAS BIBLIOGRÁFICAS

- Angular Team. (2024). Angular Documentation: NgModules vs. Standalone Components. Recuperado de: <https://angular.io/guide/ngmodules>
- Ionic Framework. (2024). Components Documentation: Build Hybrid Apps with Ionic 7. Recuperado de: <https://ionicframework.com/docs/components>
- Microsoft. (2024). TypeScript Documentation: Mastering the tsconfig.json. Recuperado de: <https://www.typescriptlang.org/docs/handbook/tsconfig-json.html>
- Nielsen Norman Group. (2022). Brutalism in Web Design: Aesthetics and Usability. Recuperado de: <https://www.nngroup.com/articles/brutalism-web-design/>
- Vercel. (2024). CI/CD Workflows for Advanced Web Applications. Recuperado de: <https://vercel.com/docs/concepts/deployments/git>